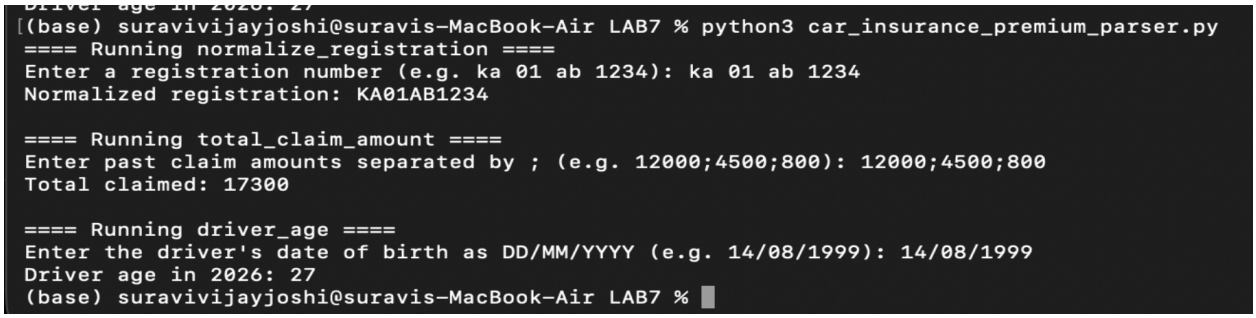
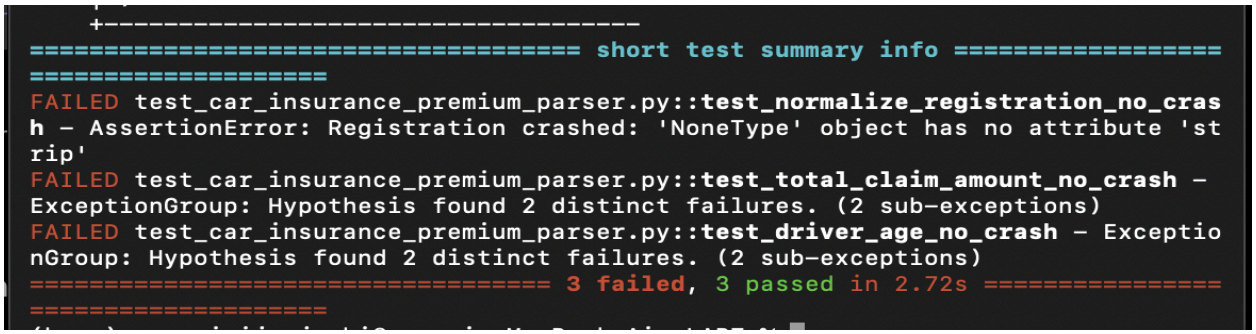
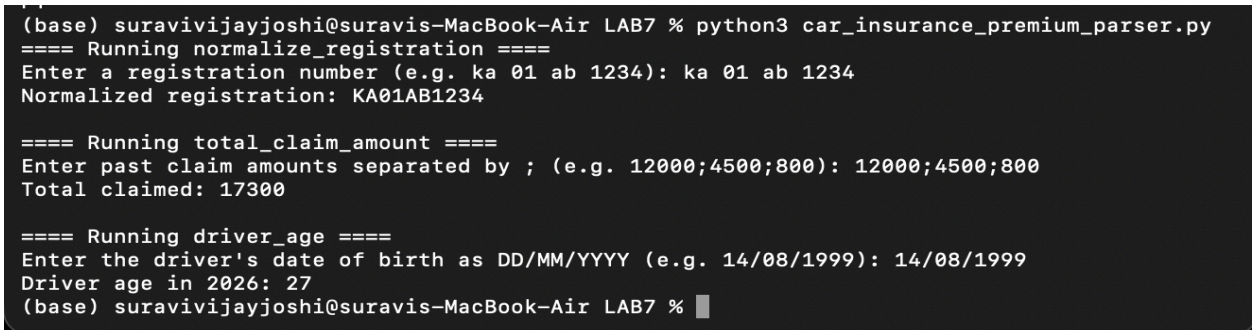
SS1: Original buggy module execution screenshot



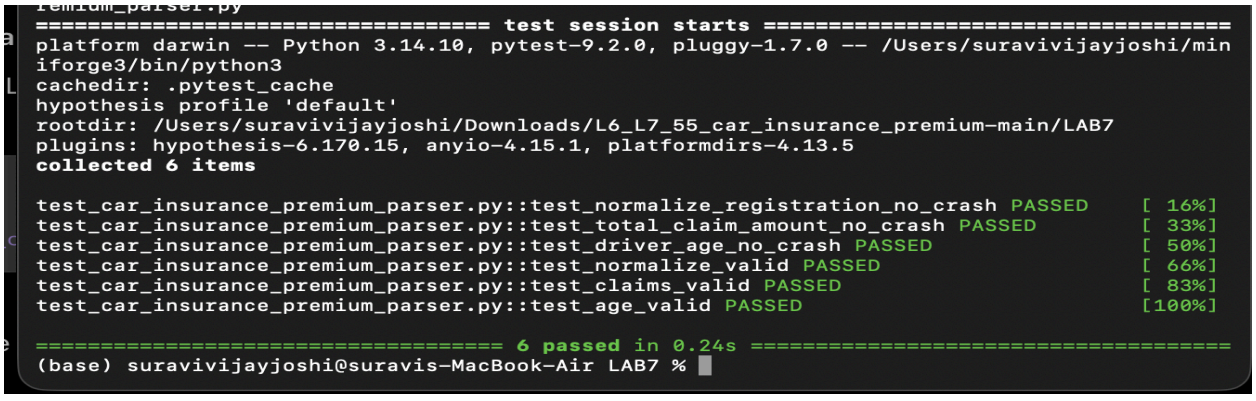

SS2: Hypothesis tests failing




SS3: Fixed Parser




SS4: Fixed Hypothesis Testing



# VIVA ANSWERS

**1. What is fuzz testing?**

Fuzz testing is a dynamic software testing technique that provides random, unexpected or malformed inputs to a program to identify crashes, bugs and vulnerabilities.

**2. What is Hypothesis?**

Hypothesis is a Python property-based testing library that automatically generates different inputs to check whether a specified property holds true.

**3. What is the difference between unit testing and fuzz testing?**

Traditional unit testing checks specific inputs and expected outputs, whereas fuzz testing automatically generates many unusual inputs to find edge cases that developers may not anticipate.

**4. What is the @given decorator?**

It tells Hypothesis to generate test inputs using the specified strategies and execute the test against those inputs.

**5. What are the three bugs identified?**

- Registration normalization crashed when given None.
- Claim calculation crashed on empty or non-numeric amounts.
- Driver age calculation crashed on malformed dates or missing date components.

**6. How did you fix them?**

I added input validation, safe type conversion and exception handling. Invalid claim entries are ignored, invalid dates return -1, and invalid registration inputs return an empty string.

**7. What is a Hypothesis strategy?**

A strategy defines the type of data Hypothesis generates. For example, st.text() generates random Unicode strings and st.none() generates None.

**8. What is shrinking in Hypothesis?**

Shrinking is the process of reducing a failing input to a smaller, simpler input that still reproduces the failure.

**9. Why is fuzz testing important?**

It helps identify unexpected edge cases, improves reliability and reduces the risk of applications crashing when users provide invalid inputs.